TCGA–DSMZ Correlation Analysis

chu25

2025-10-12

Contents

```
config <- list(                                          # Define configuration list for pipeli
  # Inputs
  tcga_se_rds   = "/home/chu25/data/tcga/ALL_TCGA_STAR_Counts_SummarizedExperiment_filtered.rds
  dsmz_rds      = "/home/chu25/data/dsmz/DSMZ_count_gene.rds",  # Path to DSMZ count data RDS
  dsmz_meta_csv = "/home/chu25/data/dsmz/DSMZ_metadata.csv",    # Path to DSMZ metadata CSV
  purity_tsv    = NULL,                                  # Optional path to purity TSV (sample, 

  # Outputs
  outdir        = "/home/chu25/dsmz/results/correlation5",  # Output directory for results
  dsmz_cache_rds = "/home/chu25/data/dsmz/DSMZ_aligned_cache.rds",  # Path to cached aligned D

  # Analysis
  var_gene_set  = "tcga_5k",                             # Gene set for variable gene selection
  min_shared    = 1000,                                  # Minimum number of shared genes requir
  batch_adjust_method = "combat_seq",                    # Batch correction method ("none", "com
  force_realign = FALSE                                  # Force realignment even if cache exist
)

# Create necessary directories
dir.create(config$outdir, showWarnings = FALSE, recursive = TRUE)  # Create output directory, 
dir.create(file.path(config$outdir, "figs"), showWarnings = FALSE, recursive = TRUE)  # Create
dir.create(file.path(config$outdir, "figs_discussion"), showWarnings = FALSE, recursive = TRUE)
knitr::opts_chunk$set(fig.path = paste0(config$outdir, "/figs-"))  # Set figure output path fo
```

```
# ============================================================================
# DATA LOADING
# ============================================================================

# Load TCGA SummarizedExperiment and extract counts
load_tcga_data <- function(file_path) {                  # Define function to load TCGA data
  cat("[INFO] Loading TCGA data...\n")                   # Print message indicating TCGA data lo
  if (!file.exists(file_path)) stop(sprintf("[ERROR] TCGA RDS not found: %s", file_path))  # S
  se <- readRDS(file_path)                               # Read TCGA SummarizedExperiment from R
```

```
  counts <- assay(se)                                    # Extract count matrix from SummarizedEx
  cat(sprintf("[INFO] TCGA dims: %d genes x %d samples\n", nrow(counts), ncol(counts)))  # Pri
  list(se = se, counts = counts)                         # Return list with SummarizedExperiment
}

# Load DSMZ raw wide table (counts + gene columns) and metadata CSV
load_dsmz_data <- function(counts_path, meta_path) {     # Define function to load DSMZ data
  cat("[INFO] Loading DSMZ data...\n")                   # Print message indicating DSMZ data lo
  if (!file.exists(counts_path)) stop(sprintf("[ERROR] DSMZ RDS not found: %s", counts_path))
  if (!file.exists(meta_path)) stop(sprintf("[ERROR] DSMZ metadata CSV not found: %s", meta_pat
  raw <- readRDS(counts_path)                            # Read DSMZ raw count data from RDS
  meta <- read.csv(meta_path)                            # Read DSMZ metadata from CSV
  stopifnot(all(c("Ensembl_ID","gene_name") %in% colnames(raw)))  # Verify required columns in
  stopifnot("sample_name" %in% names(meta))              # Verify sample_name column in metadata
  cat(sprintf("[INFO] DSMZ table: %d rows x %d cols\n", nrow(raw), ncol(raw)))  # Print dimens
  list(raw = raw, meta = meta)                           # Return list with raw counts and metad
}

# ============================================================================
# GENE ID HARMONIZATION
# ============================================================================

# Remove Ensembl version suffixes and sum duplicates
harmonize_gene_ids <- function(tcga_counts) {           # Define function to harmonize TCGA ge
  cat("[INFO] Harmonizing TCGA gene IDs...\n")          # Print message indicating gene ID harm
  g <- sub("\\..*$","", rownames(tcga_counts))          # Remove version suffixes from Ensembl
  if (any(duplicated(g))) {                            # Check for duplicate gene IDs
    tcga_counts <- rowsum(tcga_counts, g, reorder = TRUE)  # Sum counts for duplicate genes
    rownames(tcga_counts) <- sort(unique(g))           # Set unique sorted gene IDs as row nam
  } else {
    rownames(tcga_counts) <- g                         # Set harmonized gene IDs as row names
  }
  tcga_counts                                          # Return harmonized count matrix
}

# Build a numeric DSMZ count matrix from the raw wide table
build_dsmz_matrix <- function(dsmz_raw) {               # Define function to build DSMZ count
  cat("[INFO] Building DSMZ count matrix...\n")        # Print message indicating matrix const
  annot <- c("Ensembl_ID","gene_name","Ensembl_ID_with_version")  # Define annotation columns
  sample_cols <- setdiff(colnames(dsmz_raw), annot)    # Identify sample columns
  dsmz_raw[sample_cols] <- lapply(dsmz_raw[sample_cols], function(x) as.numeric(as.character(x)
  M <- as.matrix(dsmz_raw[, sample_cols, drop=FALSE])  # Create matrix from sample columns
  rownames(M) <- dsmz_raw$Ensembl_ID                   # Set Ensembl IDs as row names
  if (any(duplicated(rownames(M))))                    # Check for duplicate gene IDs
    M <- rowsum(M, rownames(M), reorder = TRUE)        # Sum counts for duplicate genes
  keep <- vapply(as.data.frame(M), function(v) any(!is.na(v)), logical(1))  # Identify columns
  if (!all(keep)) {                                    # Check if any columns are all NA
```

```
    cat(sprintf("[WARN] Dropping %d DSMZ columns that are all NA\n", sum(!keep)))  # Warn abou
    M <- M[, keep, drop=FALSE]                          # Drop all-NA columns
  }
  M                                                     # Return DSMZ count matrix
}

# Align DSMZ metadata to count matrix; write out unmatched for audit
align_metadata <- function(dsmz_meta, dsmz_counts, outdir) {  # Define function to align metad
  cat("[INFO] Aligning metadata to counts...\n")        # Print message indicating alignment
  dsmz_meta <- dsmz_meta %>% mutate(sample_id = sample_name)  # Create sample_id column from s
  matched <- intersect(dsmz_meta$sample_id, colnames(dsmz_counts))  # Find common samples betw
  md_only <- setdiff(dsmz_meta$sample_id, colnames(dsmz_counts))  # Find metadata samples not
  ct_only <- setdiff(colnames(dsmz_counts), dsmz_meta$sample_id)  # Find count samples not in
  if (length(md_only))                                  # Check if there are unmatched metadata
    write.csv(tibble(sample_name = md_only),             # Save unmatched metadata samples to C
              file.path(outdir, "unmatched_metadata_samples.csv"),
              row.names = FALSE)
  if (length(ct_only))                                  # Check if there are unmatched count sa
    write.csv(tibble(sample_name = ct_only),             # Save unmatched count samples to CSV
              file.path(outdir, "unmatched_count_columns.csv"),
              row.names = FALSE)
  dsmz_meta <- dsmz_meta %>% filter(sample_id %in% matched)  # Filter metadata to matched samp
  dsmz_counts <- dsmz_counts[, dsmz_meta$sample_id, drop=FALSE]  # Subset counts to matched sa
  stopifnot(identical(colnames(dsmz_counts), dsmz_meta$sample_id))  # Verify counts and metada
  cat(sprintf("[INFO] Matched: %d | Unmatched(meta): %d | Unmatched(counts): %d\n",
              length(matched), length(md_only), length(ct_only)))  # Print alignment summary
  list(meta = dsmz_meta, counts = dsmz_counts)          # Return list with aligned metadata and
}

# Load or create aligned DSMZ data with caching
load_or_align_dsmz_data <- function(dsmz_raw, dsmz_meta, cache_file, outdir, force_realign = F
  if (!force_realign && file.exists(cache_file)) {      # Check if cache file exists and realig
    cat("[INFO] Loading cached aligned DSMZ data from:", cache_file, "\n")  # Print message in
    cached_data <- readRDS(cache_file)                  # Load cached data
    # Verify the cached data has the expected structure
    if (all(c("meta", "counts") %in% names(cached_data)) &&
        nrow(cached_data$meta) > 0 && ncol(cached_data$counts) > 0) {
      cat(sprintf("[INFO] Cached data: %d samples, %d genes\n",
                  nrow(cached_data$meta), nrow(cached_data$counts)))  # Print cached data summ
      return(cached_data)                               # Return cached data
    } else {
      cat("[WARN] Cached data appears corrupted, realigning...\n")  # Warn about corrupted cac
    }
  }

  cat("[INFO] Building DSMZ count matrix and aligning metadata...\n")  # Print message indicat
  dsmz_counts <- build_dsmz_matrix(dsmz_raw)            # Build DSMZ count matrix
```

```r
  storage.mode(dsmz_counts) <- "double"                # Ensure double precision
  aligned_data <- align_metadata(dsmz_meta, dsmz_counts, outdir)  # Align metadata with counts

  # Save aligned data to cache
  cat("[INFO] Saving aligned DSMZ data to cache:", cache_file, "\n")  # Print message indicati
  dir.create(dirname(cache_file), showWarnings = FALSE, recursive = TRUE)  # Create cache dire
  saveRDS(aligned_data, cache_file)                    # Save aligned data to RDS
  aligned_data                                         # Return aligned data
}

# ============================================================================
# PURITY HANDLING (LOG SCALE)
# ============================================================================

# Load tumor purity vector from SummarizedExperiment or external TSV
load_purity_data <- function(tcga_se, purity_file=NULL) {  # Define function to load tumor pur
  p <- NULL                                            # Initialize purity vector as NULL
  if ("purity" %in% colnames(colData(tcga_se))) {      # Check if purity data is in SummarizedE
    p <- as.numeric(colData(tcga_se)$purity)           # Extract purity values as numeric
    names(p) <- colnames(assay(tcga_se))               # Assign sample names to purity values
    cat("[INFO] Using purity from SummarizedExperiment\n")  # Print message indicating source
  } else if (!is.null(purity_file) && file.exists(purity_file)) {  # Check if external purity
    tab <- read.delim(purity_file, stringsAsFactors=FALSE)  # Read purity TSV file
    stopifnot(all(c("sample","purity") %in% colnames(tab)))  # Verify required columns in TSV
    p <- setNames(tab$purity, tab$sample)              # Create named purity vector
    cat("[INFO] Using external purity file\n")         # Print message indicating source
  } else {
    cat("[INFO] No purity data available\n")           # Print message if no purity data
  }
  p                                                    # Return purity vector
}

# Adjust TCGA LOG matrix for purity: (1) drop negative-purity-correlated genes, (2) regress ou
# NOTE: operates on log2-CPM (not raw counts)
purity_adjust_log <- function(tcga_log, purity) {       # Define function to adjust TCGA log2-
  if (is.null(purity)) return(tcga_log)                # Return input if no purity data
  cat("[INFO] Applying purity adjustment on log2-CPM...\n")  # Print message indicating adjust
  keep_samp <- intersect(colnames(tcga_log), names(purity)[!is.na(purity)])  # Find samples wi
  M <- tcga_log[, keep_samp, drop=FALSE]               # Subset log matrix to valid samples
  pu <- purity[keep_samp]                              # Subset purity to valid samples
  if (ncol(M) < 3) {                                   # Check if enough samples for adjustment
    cat("[WARN] Too few samples with purity to adjust; returning input.\n")  # Warn if insuffi
    return(tcga_log)                                   # Return input matrix
  }
  # Identify & drop genes strongly NEG correlated with purity (likely infiltration)
  ct <- apply(M, 1, function(v) suppressWarnings(cor.test(as.numeric(v), pu, method="spearman"
  pv <- vapply(ct, `[[`, numeric(1), "p.value")        # Extract p-values
```

```
  rh <- vapply(ct, function(x) unname(x$estimate), numeric(1))  # Extract correlation coeffici
  padj <- p.adjust(pv, "BH")                                    # Adjust p-values for multiple testing
  drop <- names(which(padj < 0.01 & rh < -0.4))                 # Identify genes with significant negat
  if (length(drop)) {                                           # Check if genes need to be dropped
    cat(sprintf("[INFO] Removing %d purity-associated genes\n", length(drop)))  # Print number
    M <- M[setdiff(rownames(M), drop), , drop=FALSE]            # Remove identified genes
  }
  # Regress out infiltration (1 - purity) on the log scale
  infilt <- 1 - pu                                              # Calculate infiltration (1 - purity)
  design <- model.matrix(~ infilt)                               # Create design matrix for regression
  fit <- lmFit(M, design)                                       # Fit linear model using limma
  beta <- fit$coefficients[, "infilt", drop=FALSE]              # Extract infiltration coefficients
  Madj <- as.matrix(M) - beta %*% t(infilt)                     # Adjust matrix by subtracting infiltra
  out <- tcga_log                                               # Initialize output matrix
  common_g <- intersect(rownames(Madj), rownames(out))          # Find common genes
  out[common_g, colnames(Madj)] <- Madj[common_g, ]             # Update adjusted values
  out                                                           # Return adjusted matrix
}
```

```
# ============================================================================
# CORRELATION + MAPPING
# ============================================================================

# Compute TCGA cohort means (on whatever scale tcga_mat is; here it's log2-CPM)
compute_tcga_means <- function(tcga_se, tcga_mat) {     # Define function to compute TCGA coho
  nm <- intersect(c("project_id","study","disease_type"), colnames(colData(tcga_se)))[1]  # Se
  stopifnot(!is.na(nm))                                 # Verify metadata field exists
  labs <- as.character(colData(tcga_se)[colnames(tcga_mat), nm])  # Extract cohort labels
  stopifnot(!any(is.na(labs)))                          # Verify no missing labels
  labs <- sub("^TCGA-","", labs)                       # Remove "TCGA-" prefix from labels
  lev <- sort(unique(labs))                             # Get sorted unique cohort labels
  means <- sapply(lev, function(ct) rowMeans(tcga_mat[, labs==ct, drop=FALSE], na.rm=TRUE))  #
  colnames(means) <- lev                                # Set cohort names as column names
  list(means=means, labels=setNames(labs, colnames(tcga_mat)))  # Return means and labels
}

# Select variable genes by IQR
select_variable_genes <- function(mat, var_gene_set) {  # Define function to select variable g
  sel <- rownames(mat)                                  # Default to all genes
  if (tolower(var_gene_set) %in% c("tcga_5k","tcga_10k")) {  # Check if specific gene set is r
    n <- if (tolower(var_gene_set)=="tcga_5k") 5000 else 10000  # Set number of genes (5k or 1
    iq <- matrixStats::rowIQRs(mat)                     # Calculate interquartile range for eac
    n <- min(n, length(iq))                             # Limit to available genes
    sel <- names(sort(setNames(iq, rownames(mat)), decreasing=TRUE))[seq_len(n)]  # Select top
  }
  cat(sprintf("[INFO] Variable-gene setting: %s (%d genes used)\n", var_gene_set, length(sel)))
  sel                                                   # Return selected gene names
```

```
}

# Spearman correlations between DSMZ samples and TCGA cohort means (same gene set & scale)
compute_correlations <- function(dsmz_mat, tcga_means, dsmz_meta, sel_genes) {  # Define funct
  cat("[INFO] Computing Spearman correlations...\n")    # Print message indicating correlation
  spearman <- outer(                                   # Compute correlations using outer prod
    dsmz_meta$sample_id,                               # DSMZ sample IDs
    colnames(tcga_means),                              # TCGA cohort names
    Vectorize(function(cell, cohort) {                 # Define correlation function
      suppressWarnings(                                # Suppress warnings (e.g., for NA handl
        cor(                                           # Compute Spearman correlation
          dsmz_mat[sel_genes, cell],                   # DSMZ sample data
          tcga_means[sel_genes, cohort],               # TCGA cohort mean data
          method = "spearman",                         # Use Spearman method
          use = "pairwise.complete.obs"                # Handle missing values pairwise
        )
      )
    })
  )
  dimnames(spearman) <- list(dsmz_meta$sample_id, colnames(tcga_means))  # Set matrix dimensio
  spearman                                             # Return correlation matrix
}

# ================================================================================
# EXTERNAL ORGAN / SAMPLE → TCGA MAPPING (from CSV)
# ================================================================================

# Return the set of valid TCGA projects present in the current run
valid_tcga_projects <- function(tcga_means) colnames(tcga_means)

# Parse a user CSV to build:
#  - organ2tcga: list( organ -> character vector of TCGA project codes )
#  - sample2tcga: named character vector of per-sample TCGA overrides
# Any non-TCGA codes in the file are ignored with a warning.
parse_external_mapping <- function(csv_path, tcga_projects) {
  if (is.null(csv_path) || is.na(csv_path) || !nzchar(csv_path) || !file.exists(csv_path)) {
    return(list(organ2tcga = NULL, sample2tcga = NULL))
  }

  tab <- suppressWarnings(read.csv(csv_path, check.names = FALSE))
  std <- function(x) trimws(as.character(x))

  # Try to locate likely column names (be permissive with header variants)
  guess_col <- function(cands) {
    ix <- which(tolower(names(tab)) %in% tolower(cands))
    if (length(ix)) names(tab)[ix[1]] else NA_character_
  }
```

```r
organ_col   <- guess_col(c("organ","Organ","Tissue","tissue"))
code_col    <- guess_col(c("codes/tcga_codes","tcga_codes","codes","tcga","Project","project
sample_col  <- guess_col(c("sample_name","sample","Sample","DSMZ_Cell_line_norm","Cell_Line"

if (is.na(organ_col) || is.na(code_col)) {
  warning("[WARN] External map CSV missing 'organ' or 'codes/tcga_codes' columns; skipping ex
  return(list(organ2tcga = NULL, sample2tcga = NULL))
}

tab[[organ_col]]  <- std(tab[[organ_col]])
tab[[code_col]]   <- std(tab[[code_col]])
if (!is.na(sample_col)) tab[[sample_col]] <- std(tab[[sample_col]])

# Split multi-codes if user provided "A;B;C" or "A,B"
split_codes <- function(s) {
  if (is.na(s) || !nzchar(s)) return(character(0))
  uniq <- unique(unlist(strsplit(s, "[,; ]+")))
  uniq[ nzchar(uniq) ]
}

# Build organ → TCGA (filter only codes that are in TCGA projects)
organ_levels <- unique(tab[[organ_col]])
organ2tcga <- setNames(vector("list", length(organ_levels)), organ_levels)
for (o in organ_levels) {
  codes_raw <- unlist(lapply(tab[[code_col]][tab[[organ_col]] == o], split_codes))
  codes_tcga <- intersect(codes_raw, tcga_projects)
  codes_tcga <- unique(codes_tcga)
  if (length(codes_tcga) == 0L && length(codes_raw) > 0L) {
    bad <- setdiff(unique(codes_raw), tcga_projects)
    if (length(bad))
      message(sprintf("[INFO] Organ '%s': ignoring non-TCGA codes: %s", o, paste(bad, collaps
  }
  organ2tcga[[o]] <- codes_tcga
}

# Per-sample overrides if a valid TCGA code is present
sample2tcga <- NULL
if (!is.na(sample_col)) {
  keep_rows <- nzchar(tab[[sample_col]]) & nzchar(tab[[code_col]])
  if (any(keep_rows)) {
    s_codes <- mapply(function(s, c) {
      cs <- split_codes(c)
      cs <- intersect(cs, tcga_projects)
      if (length(cs)) cs[1] else NA_character_
    }, tab[[sample_col]][keep_rows], tab[[code_col]][keep_rows], USE.NAMES = FALSE)
    ok <- !is.na(s_codes) & nzchar(s_codes)
    if (any(ok)) sample2tcga <- setNames(s_codes[ok], tab[[sample_col]][keep_rows][ok])
```

```
    }
  }

  list(organ2tcga = organ2tcga, sample2tcga = sample2tcga)
}

# Fallback (legacy) organ → TCGA map if the CSV doesn't provide usable codes
default_organ_tcga_map <- function() {
  list(
    "Adrenal"       = c("ACC","PCPG"),
    "Adrenal/SNS"   = c("ACC","PCPG"),        # keeps SNS behavior for neuroblastoma context
    "Bladder"       = "BLCA",
    "Brain/CNS"     = c("GBM","LGG"),
    "Breast"        = "BRCA",
    "Cervix"        = "CESC",
    "Liver/Biliary" = c("LIHC","CHOL"),
    "Colon/Rectum"  = c("COAD","READ"),
    "Esophagus"     = "ESCA",
    "Head/Neck"     = "HNSC",
    "Kidney"        = c("KICH","KIRC","KIRP"),
    "Lung"          = c("LUAD","LUSC"),
    "Mesothelium"   = "MESO",
    "Ovary"         = "OV",
    "Pancreas"      = "PAAD",
    "Prostate"      = "PRAD",
    "Sarcoma"       = "SARC",
    "Skin"          = "SKCM",
    "Stomach"       = "STAD",
    "Testis"        = "TGCT",
    "Thyroid"       = "THCA",
    "Thymus"        = "THYM",
    "Uterus"        = c("UCEC","UCS"),
    "Uveal/Eye"     = "UVM",
    "Hematologic-Lymphoid" = c("DLBC"),       # DLBC exists in TCGA; others (e.g., ALL/ALCL) a
    "Hematologic-Myeloid"  = c("LAML"),       # TCGA myeloid leukemia
    "Hematologic"   = c("DLBC","LAML"),
    "Unknown"       = character(0),
    "Other"         = character(0)
  )
}

# Final mapping function: prefer external; fall back to default; allow sample overrides
build_effective_mapping <- function(tcga_means, external_csv = NULL) {
  tcga_projects <- valid_tcga_projects(tcga_means)
  ext <- parse_external_mapping(external_csv, tcga_projects)
  organ2tcga <- ext$organ2tcga
  sample2tcga <- ext$sample2tcga
```

```r
  # If external organ map is empty or all non-TCGA, merge with defaults
  if (is.null(organ2tcga) || !length(unlist(organ2tcga))) {
    organ2tcga <- default_organ_tcga_map()
  } else {
    # For any organ with no valid TCGA codes, fill from defaults if available
    def <- default_organ_tcga_map()
    for (o in names(organ2tcga)) {
      if (length(organ2tcga[[o]]) == 0L && !is.null(def[[o]])) {
        organ2tcga[[o]] <- def[[o]]
      }
    }
    # Also bring in any default organs not present in external
    for (o in setdiff(names(def), names(organ2tcga))) organ2tcga[[o]] <- def[[o]]
  }

  list(organ2tcga = organ2tcga, sample2tcga = sample2tcga)
}

# Map DSMZ samples to TCGA cohorts using:
#  (a) per-sample override when available,
#  (b) organ-constrained best- among candidates,
#  (c) otherwise global best- .
perform_organ_mapping <- function(dsmz_meta, spearman_scores, tcga_means,
                                  organ2tcga, sample2tcga = NULL) {
  cat("[INFO] Performing organ-constrained mapping (external-aware)…\n")
  corr_rows <- rownames(spearman_scores)
  corr_cols <- colnames(spearman_scores)
  tcga_codes_available <- intersect(colnames(tcga_means), corr_cols)

  dsmz_meta %>%
    mutate(
      sample_id = trimws(as.character(sample_id)),
      organ = ifelse(is.na(organ) | organ == "", "Other", trimws(as.character(organ)))
    ) %>%
    rowwise() %>%
    mutate(
      # sample-level override if provided and valid
      tcga_override = {
        if (!is.null(sample2tcga) && !is.na(sample_id) && nzchar(sample_id) &&
            (sample_id %in% names(sample2tcga))) {
          code <- sample2tcga[[sample_id]]
          if (!is.na(code) && code %in% tcga_codes_available) code else NA_character_
        } else NA_character_
      },
      tcga_candidates = list({
        cands0 <- organ2tcga[[organ]]
        if (is.null(cands0)) character(0) else intersect(cands0, tcga_codes_available)
```

```
      }),
      tcga_code = {
        sid <- sample_id
        if (!is.na(tcga_override)) {
          tcga_override
        } else if (is.na(sid) || !(sid %in% corr_rows)) {
          NA_character_
        } else {
          cand <- tcga_candidates
          if (!is.character(cand)) cand <- as.character(cand)
          if (length(cand) == 0L) {
            # fallback: global best across all TCGA cohorts
            sc_all <- spearman_scores[sid, , drop = TRUE]
            names(sc_all)[which.max(sc_all)]
          } else {
            sc <- spearman_scores[sid, cand, drop = TRUE]
            sc[!is.finite(sc)] <- -Inf
            cand[which.max(sc)]
          }
        }
      }
    ) %>%
    ungroup()
}

# Reverse mapping (TCGA project to organ)
tcga_project_to_organ_map <- function(organ2tcga = NULL) {               # Define function for
  if (is.null(organ2tcga)) organ2tcga <- default_organ_tcga_map()        # Use default if not
  organ2tcga %>%                           # Get organ-to-TCGA mapping
    enframe(name = "organ", value = "codes") %>%       # Convert to tibble with organ and code
    unnest_longer(codes, values_to = "tcga_project") %>%  # Expand codes to individual rows
    filter(!is.na(tcga_project)) %>%                    # Remove NA TCGA projects
    distinct(tcga_project, .keep_all = TRUE) %>%        # Keep unique TCGA projects
    select(tcga_project, organ)                         # Select TCGA project and organ
}
```

```
# ============================================================================
# VISUALIZATION
# ============================================================================

# Palettes for consistent plotting
get_palettes <- function() {                            # Define function to return color palet
  list(                                                 # Return list of color palettes
    okabe_ito = c("#E69F00", "#56B4E9", "#009E73", "#F0E442",
                  "#0072B2", "#D55E00", "#CC79A7", "#000000"),  # Okabe-Ito colorblind-friendl
    tol_bright = c("#4477AA", "#EE6677", "#228833", "#CCBB44",
                   "#66CCEE", "#AA3377", "#BBBBBB"),    # Tol bright palette
```

```
    tol_vibrant = c("#EE7733", "#0077BB", "#33BBEE", "#EE3377",
                    "#CC3311", "#009988", "#BBBBBB"),   # Tol vibrant palette
    two_group = c("#E69F00", "#0072B2"),               # Palette for two groups
    sequential = c("#FFFFFF", "#FFF2CC", "#FFE699", "#FFD966",
                   "#FFCC33", "#FFB300", "#E69F00", "#CC8800")  # Sequential palette
  )
}

# Create violin/heatmap/bar plots + stats tables
create_plots <- function(summary_tbl, spearman_scores, dsmz_map, outdir) {  # Define function
  cat("[INFO] Creating correlation plots...\n")       # Print message indicating plot creation
  palettes <- get_palettes()                         # Retrieve color palettes

  mean_ci95 <- function(x) {                         # Define helper function to compute mean
    x <- x[is.finite(x)]                             # Remove non-finite values
    n <- length(x); m <- mean(x); se <- stats::sd(x) / sqrt(n)  # Calculate n, mean, and stand
    data.frame(y = m, ymin = m - 1.96*se, ymax = m + 1.96*se)  # Return mean and CI bounds
  }

  fig_dir <- file.path(outdir, "figs")               # Define figure output directory
  dir.create(fig_dir, showWarnings = FALSE, recursive = TRUE)  # Create figure directory

  spearman_long <- spearman_scores %>%               # Convert correlation matrix to long for
    as.data.frame() %>%                              # Convert to data frame
    tibble::rownames_to_column("sample_id") %>%      # Add sample IDs as column
    tidyr::pivot_longer(-sample_id, names_to = "tcga_project", values_to = "rho") %>%  # Pivot
    dplyr::left_join(dsmz_map %>% dplyr::select(sample_id, organ), by = "sample_id") %>%  # Jo
    dplyr::mutate(organ = dplyr::coalesce(organ, "Other"))  # Replace NA organs with "Other"

  best_corr_data <- spearman_long %>%                # Extract highest correlation per sample
    dplyr::group_by(sample_id) %>%                   # Group by sample ID
    dplyr::slice_max(order_by = rho, n = 1, with_ties = FALSE) %>%  # Select maximum correlati
    dplyr::ungroup()                                 # Remove grouping

  organ_summary <- best_corr_data %>%                # Summarize correlations by organ
    dplyr::group_by(organ) %>%                       # Group by organ
    dplyr::summarise(                                # Compute summary statistics
      n = dplyr::n(),                                # Count samples
      mean = mean(rho, na.rm = TRUE),                # Mean correlation
      sd = sd(rho, na.rm = TRUE),                    # Standard deviation
      se = sd/sqrt(n),                               # Standard error
      ci95_low = mean - 1.96*se,                     # Lower 95% CI
      ci95_high = mean + 1.96*se,                    # Upper 95% CI
      .groups = "drop"                               # Drop grouping
    ) %>%
    dplyr::arrange(dplyr::desc(mean))                # Sort by mean correlation
  readr::write_csv(organ_summary, file.path(fig_dir, "violin_rho_by_organ_summary.csv"))  # Sa
```

```r
kw <- rstatix::kruskal_test(best_corr_data, rho ~ organ)  # Perform Kruskal-Wallis test
dunn <- rstatix::dunn_test(best_corr_data, rho ~ organ, p.adjust.method = "BH") %>%  # Perfo
  rstatix::add_significance("p.adj")                # Add significance symbols
dunn_sig <- dunn %>% dplyr::filter(p.adj < 0.05)    # Filter significant comparisons
if (nrow(dunn_sig) > 0) {                           # Check if significant comparisons exist
  dunn_sig <- rstatix::add_xy_position(dunn_sig, x = "organ", data = best_corr_data, step.inc
}

organs <- sort(unique(best_corr_data$organ))        # Get sorted unique organs
n_organs <- length(organs)                          # Count number of organs
organ_colors <- if (n_organs <= 8) palettes$okabe_ito[1:n_organs]  # Use Okabe-Ito palette f
                else c(palettes$okabe_ito, palettes$tol_bright, palettes$tol_vibrant)[1:n_org
names(organ_colors) <- organs                       # Assign organ names to colors

kw_p <- kw$p[[1]]                                   # Extract Kruskal-Wallis p-value
subtitle_txt <- paste0("Kruskal-Wallis p = ", formatC(kw_p, format = "e", digits = 2))  # Fo

p1 <- ggplot(best_corr_data, aes(x = organ, y = rho)) +  # Initialize violin plot
  geom_violin(aes(fill = organ), trim = FALSE, alpha = 0.6, color = "black", size = 0.5) +
  geom_jitter(aes(color = organ, shape = organ), width = 0.15, alpha = 0.7, size = 1.5) +  #
  stat_summary(fun.data = mean_ci95, geom = "pointrange", color = "black", size = 0.8, fatter
  scale_fill_manual(values = organ_colors, guide = "none") +  # Set fill colors, hide legend
  scale_color_manual(values = organ_colors, guide = "none") +  # Set point colors, hide leges
  scale_shape_manual(values = rep(c(16, 17, 15, 18, 8, 4, 3, 7), length.out = n_organs), guid
  theme_bw(base_size = 12) +                        # Use black-and-white theme
  theme(                                            # Customize theme
    axis.text.x = element_text(angle = 45, hjust = 1, size = 11),  # Rotate x-axis labels
    axis.text.y = element_text(size = 11),          # Set y-axis text size
    axis.title = element_text(size = 12, face = "bold"),  # Bold axis titles
    plot.title = element_text(size = 14, face = "bold"),  # Bold plot title
    plot.subtitle = element_text(size = 11),        # Set subtitle size
    panel.grid.minor = element_blank(),             # Remove minor grid lines
    panel.grid.major.x = element_blank()            # Remove major x grid lines
  ) +
  labs(                                             # Add plot labels
    x = "Organ/Tissue Type",                        # X-axis label
    y = "Best Spearman  ",                          # Y-axis label
    title = "Distribution of Best DSMZ→TCGA Correlations by Organ/Tissue",  # Plot title
    subtitle = subtitle_txt                         # Plot subtitle
  )
if (nrow(dunn_sig) > 0) {                           # Check if significant comparisons exist
  p1 <- p1 + ggpubr::stat_pvalue_manual(dunn_sig, label = "p.adj.signif", hide.ns = TRUE, tip
}
print(p1)                                           # Print violin plot
ggsave(file.path(fig_dir, "violin_rho_by_organ.pdf"), p1, width = 12, height = 7)  # Save vi

# Heatmap of top 25
```

```r
topN <- head(dplyr::arrange(summary_tbl, dplyr::desc(best_rho_overall))$sample_id, 25)  # Se
hm <- spearman_scores[topN, , drop = FALSE]          # Subset correlation matrix for top 25
colors_div <- rev(RColorBrewer::brewer.pal(11, "RdYlBu"))  # Create reversed RdYlBu palette
hp <- pheatmap::pheatmap(                            # Create heatmap
  hm, cluster_rows = TRUE, cluster_cols = TRUE,       # Enable clustering
  color = colorRampPalette(colors_div)(100),          # Use smooth color gradient
  main = "Top 25 DSMZ Samples: Spearman Correlation with TCGA Cohorts",  # Heatmap title
  fontsize = 9, fontsize_row = 7, fontsize_col = 8,  # Set font sizes
  border_color = "grey60", silent = TRUE             # Set border color, suppress output
)
grid::grid.newpage(); grid::grid.draw(hp$gtable)     # Render heatmap
pdf(file.path(fig_dir, "heatmap_top25.pdf"), width = 12, height = 10)  # Open PDF device
grid::grid.newpage(); grid::grid.draw(hp$gtable)     # Draw heatmap in PDF
dev.off()                                            # Close PDF device

# Stacked bar: DSMZ organ vs assigned TCGA cohort
assignment_data <- dsmz_map %>%                       # Prepare data for bar chart
  dplyr::mutate(tcga_assignment = ifelse(is.na(tcga_code) | tcga_code == "", "Unassigned", tc
  dplyr::count(organ, tcga_assignment, name = "count")  # Count samples by organ and TCGA as
totals <- assignment_data %>% dplyr::group_by(organ) %>% dplyr::summarise(total = sum(count)
organ_order <- totals$organ                          # Define organ order
assignment_data$organ <- factor(assignment_data$organ, levels = organ_order)  # Convert orga
totals$organ <- factor(totals$organ, levels = organ_order)  # Convert totals organ to factor
assignment_labels <- assignment_data %>%              # Calculate label positions
  dplyr::group_by(organ) %>%                         # Group by organ
  dplyr::arrange(organ, tcga_assignment, .by_group = TRUE) %>%  # Sort within groups
  dplyr::mutate(pos = cumsum(count) - count/2) %>%  # Calculate midpoint for labels
  dplyr::ungroup()                                   # Remove grouping
tcga_assignments <- sort(unique(assignment_data$tcga_assignment))  # Get unique TCGA assignm
n_tcga <- length(tcga_assignments)                   # Count TCGA assignments
tcga_colors <- c(palettes$okabe_ito, palettes$tol_bright, palettes$tol_vibrant)[1:n_tcga]  #
names(tcga_colors) <- tcga_assignments               # Name colors by assignments
label_df <- dplyr::filter(assignment_labels, count >= 2)  # Filter labels for segments with
assignment_data <- assignment_data %>% dplyr::mutate(tcga_assignment = factor(tcga_assignment
label_df <- label_df %>% dplyr::mutate(tcga_assignment = factor(tcga_assignment, levels = tcg

p3 <- ggplot(assignment_data, aes(x = organ, y = count, fill = tcga_assignment)) +  # Initia
  geom_col(color = "grey15", linewidth = 0.2) +      # Add bars with grey outline
  geom_text(data = label_df, inherit.aes = FALSE, aes(x = organ, y = pos, label = count), siz
  geom_text(data = totals, inherit.aes = FALSE, aes(x = organ, y = total, label = total), vju
  scale_fill_manual(values = tcga_colors, name = "TCGA Assignment") +  # Set fill colors
  scale_y_continuous(expand = expansion(mult = c(0.02, 0.12))) +  # Adjust y-axis spacing
  theme_bw(base_size = 11) +                         # Use black-and-white theme
  labs(x = "Organ/Tissue Type", y = "Number of Cell Lines", title = "DSMZ Cell Line Assignmen
  theme(axis.text.x = element_text(angle = 45, hjust = 1), legend.position = "bottom") +  #
  guides(fill = guide_legend(nrow = 2, byrow = TRUE))  # Use 2-row legend
print(p3)                                            # Print bar chart
```

```r
  ggsave(file.path(fig_dir, "organ_tcga_assignment.pdf"), p3, width = 14, height = 9)  # Save

  readr::write_csv(kw, file.path(fig_dir, "violin_kw_results.csv"))  # Save Kruskal-Wallis res
  readr::write_csv(dunn, file.path(fig_dir, "violin_dunn_results.csv"))  # Save Dunn test resu
  cat("[INFO] Plots saved to: ", fig_dir, "\n")       # Print message indicating plot save loc
}

# ============================================================================
# DISCUSSION PLOTS (3-view analysis)
# ============================================================================

create_discussion_plots <- function(summary_tbl, spearman_scores, dsmz_map, raw_meta, outdir) {
  cat("[INFO] Creating 3-view discussion plots...\n")
  palettes <- get_palettes()
  fig_dir <- file.path(outdir, "figs_discussion")
  dir.create(fig_dir, showWarnings = FALSE, recursive = TRUE)

  save_both <- function(p, path, width, height) {
    ggsave(paste0(path, ".pdf"), p, width = width, height = height)
    ggsave(paste0(path, ".png"), p, width = width, height = height, dpi = 300)
  }
  `%nin%` <- function(x, y) !(x %in% y)

  # ---------- Common data ----------
  # Long correlations
  spearman_long <- as.data.frame(spearman_scores) %>%
    tibble::rownames_to_column("sample_id") %>%
    tidyr::pivot_longer(-sample_id, names_to = "tcga_project", values_to = "rho")

  # Best overall (global)
  best_overall <- spearman_long %>%
    dplyr::group_by(sample_id) %>%
    dplyr::slice_max(order_by = rho, n = 1, with_ties = FALSE) %>%
    dplyr::ungroup()

  # Organ palette
  organs <- sort(unique(dsmz_map$organ %||% "Other"))
  n_org <- length(organs)
  organ_colors <- if (n_org <= 8) palettes$okabe_ito[1:n_org]
                  else c(palettes$okabe_ito, palettes$tol_bright, palettes$tol_vibrant)[1:n_org
  names(organ_colors) <- organs

  # =============== (1) Direct description of DSMZ cell lines ================
  # Choose descriptor columns present, and build sample_id robustly
  have_cols <- intersect(
    c("sample_name","Cell_Line","group1","group2","Disease","organ",
      "Origin","Stage","Primary.Metastatic","MYCN.amplification"),
```

```
  names(raw_meta)
)

descriptor <- raw_meta %>%
  # keep descriptor columns but drop 'organ' to avoid collision
  dplyr::select(dplyr::all_of(setdiff(have_cols, "organ"))) %>%
  dplyr::mutate(
    sample_id = dplyr::coalesce(.data$sample_name, .data$Cell_Line)
  ) %>%
  # keep organ from dsmz_map only
  dplyr::right_join(
    dsmz_map %>% dplyr::select(sample_id, organ),
    by = "sample_id"
  ) %>%
  dplyr::relocate(sample_id, organ)

# Write the descriptor table
readr::write_csv(descriptor, file.path(fig_dir, "dsmz_descriptor_table.csv"))

# (1a) Counts by organ (optionally fill by group2 if present)
if ("group2" %in% names(descriptor)) {
  p_desc_counts <- ggplot(descriptor, aes(x = organ, fill = group2)) +
    geom_bar(color = "grey20") +
    scale_fill_manual(values = c(palettes$okabe_ito, palettes$tol_bright, palettes$tol_vibra
                      guide = guide_legend(title = "group2")) +
    theme_bw(base_size = 14) +
    theme(axis.text.x = element_text(angle = 35, hjust = 1)) +
    labs(title = "DSMZ cell lines by organ", x = NULL, y = "Count")
} else {
  p_desc_counts <- ggplot(descriptor, aes(x = organ)) +
    geom_bar(fill = palettes$okabe_ito[2], color = "grey20") +
    theme_bw(base_size = 14) +
    theme(axis.text.x = element_text(angle = 35, hjust = 1)) +
    labs(title = "DSMZ cell lines by organ", x = NULL, y = "Count")
}
save_both(p_desc_counts, file.path(fig_dir, "01a_dsmz_counts_by_organ"), 11, 6)

# (1b) Metadata tile plot (top 30 samples for readability)
tile_cols <- c("group1","group2","Disease","Origin","Stage","Primary.Metastatic","MYCN.ampli
tile_cols <- tile_cols[tile_cols %in% names(descriptor)]
if (length(tile_cols) > 0) {
  topN <- head(sort(unique(descriptor$sample_id)), 30)
  tile_df <- descriptor %>%
    dplyr::filter(sample_id %in% topN) %>%
    tidyr::pivot_longer(cols = dplyr::all_of(tile_cols), names_to = "field", values_to = "va
    dplyr::mutate(sample_id = factor(sample_id, levels = rev(unique(sample_id))),
                  field = factor(field, levels = tile_cols))
```

```r
  p_tiles <- ggplot(tile_df, aes(field, sample_id, fill = value)) +
    geom_tile(color = "white") +
    scale_fill_discrete(na.translate = TRUE) +
    theme_bw(base_size = 12) +
    theme(axis.text.x = element_text(angle = 35, hjust = 1),
          legend.position = "right") +
    labs(title = "DSMZ metadata (top 30 samples)", x = NULL, y = NULL, fill = NULL)
  save_both(p_tiles, file.path(fig_dir, "01b_dsmz_metadata_tiles"), 12, 10)
}

# ===== (2) Mapping to TCGA using DSMZ organ → TCGA code constraints =========
# Here we use dsmz_map$tcga_code (already organ-constrained in your pipeline)
constrained_assign <- dsmz_map %>%
  dplyr::mutate(tcga_assignment = dplyr::coalesce(tcga_code, "Unassigned")) %>%
  dplyr::count(organ, tcga_assignment, name = "count")

# Collapse small cohorts to "Other cohorts" for readability
top_cohorts_constr <- constrained_assign %>%
  dplyr::group_by(tcga_assignment) %>%
  dplyr::summarise(total = sum(count), .groups = "drop") %>%
  dplyr::arrange(dplyr::desc(total)) %>%
  dplyr::slice(1:8) %>% dplyr::pull(tcga_assignment)

constrained_assign$tcga_assignment_collapsed <- ifelse(
  constrained_assign$tcga_assignment %in% top_cohorts_constr,
  constrained_assign$tcga_assignment, "Other cohorts"
)
constrained_assign$organ <- factor(constrained_assign$organ, levels = organs)

p_constrained <- ggplot(constrained_assign %>%
                          dplyr::group_by(organ, tcga_assignment_collapsed) %>%
                          dplyr::summarise(count = sum(count), .groups = "drop"),
                        aes(x = organ, y = count, fill = tcga_assignment_collapsed)) +
  geom_col(color = "grey15", linewidth = 0.2) +
  theme_bw(base_size = 14) +
  theme(axis.text.x = element_text(angle = 35, hjust = 1),
        legend.position = "bottom") +
  labs(title = "TCGA mapping (DSMZ organ-constrained)", x = NULL, y = "Cell lines", fill = "
save_both(p_constrained, file.path(fig_dir, "02_tcga_mapping_constrained"), 12, 7)

readr::write_csv(constrained_assign, file.path(fig_dir, "02_tcga_mapping_constrained_counts.

# ====== (3) Mapping strictly by global best TCGA cohort (max ) =============
best_global <- best_overall %>%
  dplyr::left_join(dsmz_map %>% dplyr::select(sample_id, organ), by = "sample_id") %>%
  dplyr::mutate(organ = dplyr::coalesce(organ, "Other"),
                best_tcga_overall = tcga_project)
```

```r
  global_assign <- best_global %>%
    dplyr::count(organ, best_tcga_overall, name = "count")

  # Collapse small cohorts
  top_cohorts_global <- global_assign %>%
    dplyr::group_by(best_tcga_overall) %>%
    dplyr::summarise(total = sum(count), .groups = "drop") %>%
    dplyr::arrange(dplyr::desc(total)) %>%
    dplyr::slice(1:8) %>% dplyr::pull(best_tcga_overall)

  global_assign$cohort_collapsed <- ifelse(
    global_assign$best_tcga_overall %in% top_cohorts_global,
    global_assign$best_tcga_overall, "Other cohorts"
  )
  global_assign$organ <- factor(global_assign$organ, levels = organs)

  p_global_bar <- ggplot(global_assign %>%
                           dplyr::group_by(organ, cohort_collapsed) %>%
                           dplyr::summarise(count = sum(count), .groups = "drop"),
                         aes(x = organ, y = count, fill = cohort_collapsed)) +
    geom_col(color = "grey15", linewidth = 0.2) +
    theme_bw(base_size = 14) +
    theme(axis.text.x = element_text(angle = 35, hjust = 1),
          legend.position = "bottom") +
    labs(title = "TCGA mapping by global best  (ignores organ rules)",
         x = NULL, y = "Cell lines", fill = "TCGA cohort")
  save_both(p_global_bar, file.path(fig_dir, "03a_tcga_mapping_global_best_bar"), 12, 7)

  # Top-15 lollipop of highest
  top15 <- best_overall %>%
    dplyr::arrange(dplyr::desc(rho)) %>%
    dplyr::slice(1:15) %>%
    dplyr::left_join(dsmz_map %>% dplyr::select(sample_id, organ), by = "sample_id") %>%
    dplyr::mutate(sample_id = factor(sample_id, levels = rev(sample_id)))
  p_global_lolli <- ggplot(top15, aes(x = rho, y = sample_id)) +
    geom_segment(aes(x = 0, xend = rho, y = sample_id, yend = sample_id),
                 linewidth = 1, color = "grey70") +
    geom_point(aes(color = organ), size = 3) +
    scale_color_manual(values = organ_colors, name = "Organ") +
    theme_bw(base_size = 14) +
    labs(title = "Top 15 samples by best  (global)", x = "Best Spearman ", y = NULL)
  save_both(p_global_lolli, file.path(fig_dir, "03b_tcga_mapping_global_best_lollipop"), 10, 8)

  readr::write_csv(global_assign, file.path(fig_dir, "03_tcga_mapping_global_best_counts.csv"))

  cat("[INFO] Discussion plots saved to: ", fig_dir, "\n")
}
```

```r
# ============================================================================
# PCA + BATCH EFFECT ASSESSMENT
# ============================================================================

# PCA + R² of PCs ~ dataset indicator
pc_dataset_s_ <- function(M_log, dataset_factor, top_n = 10, outdir = NULL, title_tag = "datase
  if (!is.matrix(M_log) && !is.data.frame(M_log)) stop("M_log must be a matrix or data frame")
  if (ncol(M_log) != length(dataset_factor)) stop("Length mismatch between M_log columns and da
  palettes <- get_palettes()                              # Retrieve color palettes

  pc <- prcomp(t(M_log), scale. = TRUE)                   # Perform PCA on transposed log-transfor
  var_explained <- (pc$sdev^2) / sum(pc$sdev^2)           # Calculate proportion of variance expla

  if (is.null(top_n) || top_n > ncol(pc$x)) {             # Check if top_n is invalid
    cum_var <- cumsum(var_explained)                      # Calculate cumulative variance
    top_n <- which.max(cum_var >= 0.9)                    # Select PCs explaining 90% variance
    top_n <- min(top_n, ncol(pc$x))                       # Limit to available PCs
  }

  get_r2 <- function(y) unname(summary(lm(y ~ dataset_factor))$r.squared)  # Define function t
  pcs_to_check <- seq_len(min(top_n, ncol(pc$x)))         # Define PCs to analyze
  r2_values <- sapply(pcs_to_check, function(i) get_r2(pc$x[, i]))  # Compute R² for each PC

  df_r2 <- data.frame(                                    # Create R² summary table
    PC = paste0("PC", pcs_to_check),                      # PC names
    R2 = r2_values,                                       # R² values
    VariancePct = var_explained[pcs_to_check] * 100       # Variance percentage
  )
  df_r2$PC <- factor(df_r2$PC, levels = df_r2$PC)         # Convert PC to factor for plotting

  p1 <- ggplot(df_r2, aes(x = PC, y = R2, fill = VariancePct)) +  # Initialize R² bar plot
    geom_col(color = "black", size = 0.3) +               # Add bars with black outline
    scale_fill_viridis_c(option = "plasma", name = "Variance explained (%)",  # Use viridis co
                         limits = c(0, max(var_explained) * 100)) +
    coord_cartesian(ylim = c(0, 1)) +                     # Fix y-axis range
    theme_bw(base_size = 12) +                            # Use black-and-white theme
    theme(axis.text.x = element_text(angle = 45, hjust = 1),  # Rotate x-axis labels
          axis.title = element_text(face = "bold"),       # Bold axis titles
          plot.title = element_text(face = "bold"),       # Bold plot title
          legend.position = "right") +                    # Place legend on right
    labs(title = "Variance in PCs Explained by Dataset Factor",  # Add labels
         subtitle = sprintf("Top %d PCs (%.1f%% cumulative variance)", top_n, sum(var_explained
         x = "Principal Component",
         y = expression(R^2 ~ "(dataset factor)"))

  pc_df <- as.data.frame(pc$x[, 1:2])                     # Create data frame with PC1 and PC2
```

```r
  pc_df$Group <- dataset_factor                          # Add dataset factor
  pc1_var <- round(var_explained[1] * 100, 1)            # Calculate PC1 variance percentage
  pc2_var <- round(var_explained[2] * 100, 1)            # Calculate PC2 variance percentage
  group_levels <- unique(dataset_factor)                 # Get unique dataset groups
  group_colors <- setNames(get_palettes()$two_group[seq_along(group_levels)], group_levels)  #

  p2 <- ggplot(pc_df, aes(x = PC1, y = PC2, color = Group, shape = Group)) +  # Initialize PCA
    geom_point(size = 2.5, alpha = 0.8, stroke = 0.5) +  # Add points
    stat_ellipse(aes(fill = Group), geom = "polygon", alpha = 0.15, linetype = "dashed", size =
    scale_color_manual(values = group_colors, name = title_tag) +  # Set point colors
    scale_fill_manual(values = group_colors, guide = "none") +  # Set ellipse fill, hide legen
    scale_shape_manual(values = c(16, 17)[seq_along(group_levels)], name = title_tag) +  # Set
    theme_bw(base_size = 12) +                           # Use black-and-white theme
    theme(legend.position = "right",                      # Place legend on right
          axis.title = element_text(face = "bold"),      # Bold axis titles
          plot.title = element_text(face = "bold"),      # Bold plot title
          panel.grid.minor = element_blank()) +          # Remove minor grid lines
    labs(title = paste("PCA: PC1 vs PC2 -", title_tag, "Comparison"),  # Add labels
         x = paste0("PC1 (", pc1_var, "% variance)"),
         y = paste0("PC2 (", pc2_var, "% variance)")) +
    coord_fixed(ratio = 1)                               # Set equal aspect ratio

  combined_plot <- p1 / p2 + patchwork::plot_layout(heights = c(1, 1.2))  # Combine plots vert
  print(combined_plot)                                   # Print combined plot

  if (!is.null(outdir)) {                                # Check if output directory is specified
    fig_dir <- file.path(outdir, "figs")                 # Define figure directory
    dir.create(fig_dir, showWarnings = FALSE, recursive = TRUE)  # Create figure directory
    ggsave(file.path(fig_dir, paste0("PCA_s_", title_tag, ".pdf")), combined_plot, width = 10,
    write.csv(df_r2, file.path(fig_dir, paste0("PCA_s_", title_tag, "_R2_table.csv")), row.name
  }
  return(list(r2_table = df_r2, pc = pc, plot = combined_plot,  # Return PCA results
              summary = list(total_variance = sum(var_explained[pcs_to_check]),
                             significant_pcs = sum(r2_values > 0.1))))
}

# Scree plot
make_scree_plot_ <- function(pc, outdir, title_tag, k = 20) {  # Define function to create scr
  palettes <- get_palettes()                             # Retrieve color palettes
  var_expl <- (pc$sdev^2) / sum(pc$sdev^2)              # Calculate variance explained
  k <- min(k, length(var_expl))                          # Limit k to available PCs
  df <- data.frame(                                      # Create data frame for plotting
    PC = factor(paste0("PC", seq_len(k)), levels = paste0("PC", seq_len(k))),  # PC names
    Var = var_expl[seq_len(k)] * 100,                    # Variance percentage
    Cum = cumsum(var_expl[seq_len(k)]) * 100             # Cumulative variance percentage
  )
  p <- ggplot(df, aes(PC, Var)) +                        # Initialize scree plot
```

```r
    geom_col(fill = palettes$okabe_ito[5], color = "black", size = 0.4, alpha = 0.85) +  # Add
    geom_line(aes(y = Cum, group = 1), color = palettes$okabe_ito[6], size = 1.5) +  # Add cum
    geom_point(aes(y = Cum), color = palettes$okabe_ito[6], fill = "white", shape = 21, size =
    theme_bw(base_size = 12) +                          # Use black-and-white theme
    theme(axis.text.x = element_text(angle = 45, hjust = 1),  # Rotate x-axis labels
          axis.title = element_text(face = "bold"),  # Bold axis titles
          plot.title = element_text(face = "bold"),  # Bold plot title
          plot.caption = element_text(hjust = 0, face = "italic"),  # Italic caption
          panel.grid.minor = element_blank()) +       # Remove minor grid lines
    labs(title = paste("Scree Plot -", title_tag),    # Add labels
         x = "Principal Component",
         y = "Variance Explained (%)",
         caption = "Blue bars: per-PC variance; Orange line/points: cumulative variance")
  if (!is.null(outdir)) {                             # Check if output directory is specified
    dir.create(file.path(outdir, "figs"), showWarnings = FALSE, recursive = TRUE)  # Create fi
    ggsave(file.path(outdir, "figs", paste0("scree_", title_tag, ".pdf")), p, width = 10, heig
  }
  return(p)                                           # Return scree plot
}

# PC ~ covariate R² heatmap
pc_covariate_r2_matrix_ <- function(pc, sample_df, covariates, n_pcs = 10, outdir = NULL, tag =
  stopifnot(is.list(pc), !is.null(pc$x))              # Validate PCA object
  stopifnot("sample" %in% colnames(sample_df))         # Validate sample column in metadata
  rn <- rownames(pc$x)                                # Get PCA sample IDs
  if (is.null(rn)) stop("pc$x must have rownames (sample IDs).")  # Stop if no sample IDs
  md <- sample_df[match(rn, sample_df$sample), , drop = FALSE]  # Align metadata to PCA sample
  k <- min(n_pcs, ncol(pc$x)); pcs <- paste0("PC", seq_len(k))  # Define PCs to analyze
  Y <- pc$x[, seq_len(k), drop = FALSE]               # Extract PC scores
  covariates <- covariates[covariates %in% colnames(md)]  # Filter valid covariates
  if (!length(covariates)) stop("No valid covariates found in metadata.")  # Stop if no valid
  r2_mat <- matrix(NA_real_, nrow = k, ncol = length(covariates), dimnames = list(pcs, covariat
  for (j in seq_along(covariates)) {                   # Loop through covariates
    v <- md[[covariates[j]]]                           # Extract covariate values
    if (is.character(v) || is.logical(v)) v <- factor(v)  # Convert to factor if needed
    for (i in seq_len(k)) {                            # Loop through PCs
      y <- Y[, i]; ok <- is.finite(y) & !is.na(v)      # Identify valid observations
      if (!any(ok)) next                               # Skip if no valid observations
      if (is.factor(v) && nlevels(droplevels(v[ok])) < 2) next  # Skip if factor has <2 levels
      fit <- try(lm(y[ok] ~ v[ok]), silent = TRUE)     # Fit linear model
      if (!inherits(fit, "try-error")) r2_mat[i, j] <- summary(fit)$r.squared  # Store R²
    }
  }
  if (!is.null(outdir)) {                              # Check if output directory is specified
    dir.create(outdir, showWarnings = FALSE, recursive = TRUE)  # Create output directory
    write.csv(as.data.frame(r2_mat), file.path(outdir, paste0("PC_covariate_R2_matrix_", tag,
  }
```

```r
  r2_df <- as.data.frame(r2_mat) %>% tibble::rownames_to_column("PC") %>%  # Convert R² matrix
    tidyr::pivot_longer(-PC, names_to = "Covariate", values_to = "R2")
  p <- ggplot(r2_df, aes(Covariate, PC, fill = R2)) +  # Initialize heatmap
    geom_tile(color = "white", size = 0.5) +          # Add tiles
    scale_fill_viridis_c(option = "plasma", na.value = "grey95", limits = c(0, 1), name = "R²")
    theme_bw(base_size = 11) +                        # Use black-and-white theme
    theme(axis.text.x = element_text(angle = 45, hjust = 1, size = 10),  # Customize theme
          axis.text.y = element_text(size = 10),
          axis.title = element_text(face = "bold", size = 11),
          plot.title = element_text(face = "bold", size = 12),
          panel.grid = element_blank(),
          legend.position = "right") +
    labs(title = paste0("PC ~ covariate R² (", tag, ")"),  # Add labels
         x = "Covariate", y = "Principal Component")
  if (isTRUE(show_plot)) print(p)                     # Print plot if requested
  if (!is.null(outdir)) ggsave(file.path(outdir, paste0("PC_covariate_R2_matrix_", tag, ".pdf")
  invisible(list(r2 = r2_mat, plot = p))              # Return R² matrix and plot
}

# ============================================================================
# OUTPUT HELPERS
# ============================================================================
save_results <- function(spearman_scores, summary_tbl, dsmz_map, outdir) {  # Define function
  cat("[INFO] Saving correlation tables...\n")       # Print message indicating saving
  all_wide <- as.data.frame(spearman_scores) %>% tibble::rownames_to_column("sample_id")  # Co
  all_long <- all_wide %>% pivot_longer(-sample_id, names_to = "tcga_cohort", values_to = "rho"
  write.csv(all_wide, file.path(outdir, "spearman_all_wide.csv"), row.names = FALSE)  # Save w
  write.csv(all_long, file.path(outdir, "spearman_all_long.csv"), row.names = FALSE)  # Save l
  saveRDS(spearman_scores, file.path(outdir, "spearman_all.rds"))  # Save correlation matrix a
  write.csv(summary_tbl, file.path(outdir, "spearman_best_by_sample.csv"), row.names = FALSE)
  write.csv(dsmz_map %>% select(sample_id, organ, tcga_code), file.path(outdir, "dsmz_tcga_mapp
}

save_debug_info <- function(tcga_counts, dsmz_raw, common_genes, outdir) {  # Define function
  writeLines(utils::capture.output(head(rownames(tcga_counts))), file.path(outdir, "head_tcga_g
  writeLines(utils::capture.output(head(dsmz_raw$Ensembl_ID)), file.path(outdir, "head_dsmz_ger
  writeLines(common_genes[1:min(100, length(common_genes))], file.path(outdir, "common_genes_he
  sink(file.path(outdir, "sessionInfo.txt")); print(sessionInfo()); sink()  # Save session inf
}

# ============================================================================
# MAIN PIPELINE
# ============================================================================

main <- function() {                                  # Define main pipeline function
  cat("[INFO] Starting pipeline...\n")                # Print message indicating pipeline star
```

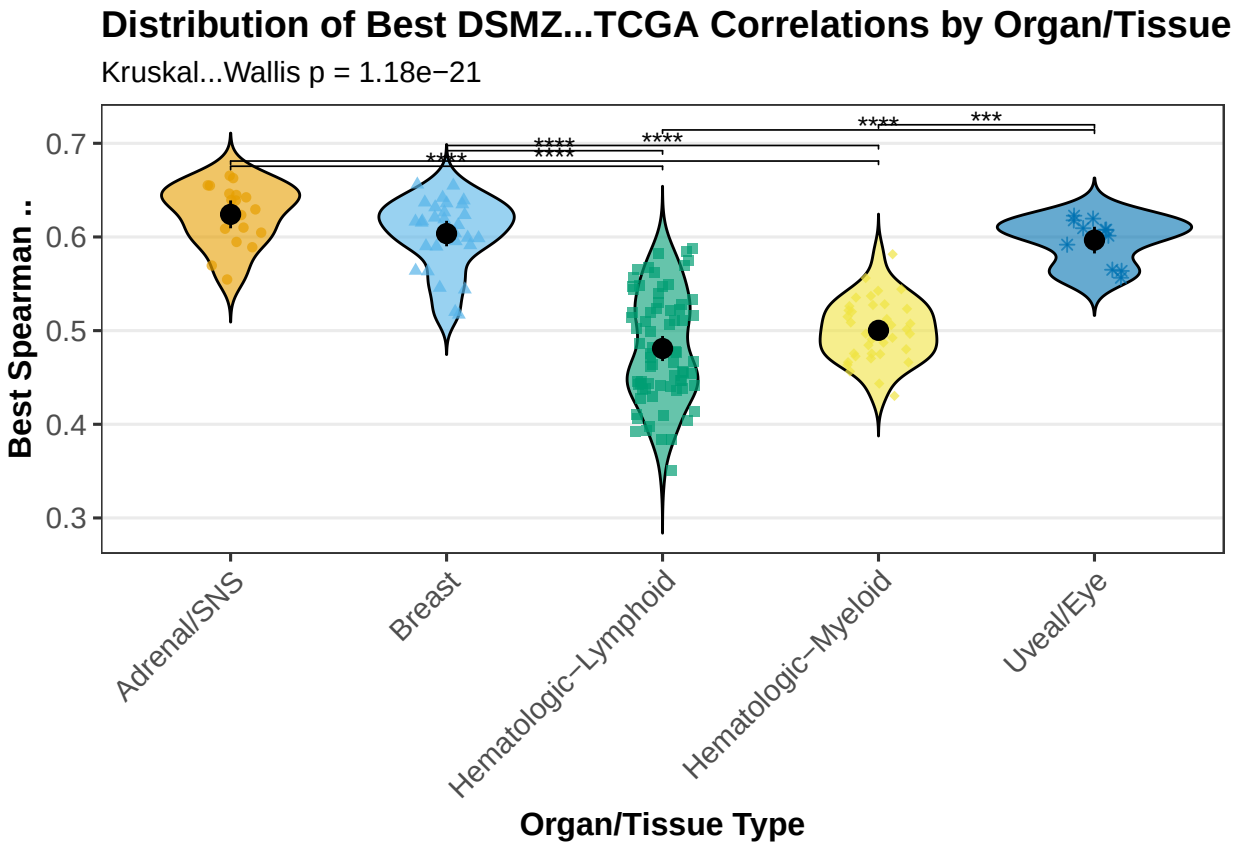
```
# Load input datasets
tcga <- load_tcga_data(config$tcga_se_rds)          # Load TCGA data
dsmz <- load_dsmz_data(config$dsmz_rds, config$dsmz_meta_csv)  # Load DSMZ data

# Harmonize & build matrices
tcga_counts <- harmonize_gene_ids(tcga$counts); storage.mode(tcga_counts) <- "double"  # Har

# Load or create aligned DSMZ data with caching
aligned_dsmz <- load_or_align_dsmz_data(dsmz$raw, dsmz$meta, config$dsmz_cache_rds, config$ou
dsmz_meta <- aligned_dsmz$meta                      # Extract aligned metadata
dsmz_counts <- aligned_dsmz$counts                  # Extract aligned counts
stopifnot("organ" %in% colnames(dsmz_meta))         # Verify organ column exists

# Common genes
common <- intersect(rownames(tcga_counts), rownames(dsmz_counts))  # Find common genes
cat(sprintf("[INFO] Shared genes: %d\n", length(common)))  # Print number of shared genes
if (length(common) < config$min_shared) cat("[WARN] Few shared genes; check Ensembl IDs\n")
tcga_counts <- tcga_counts[common, , drop=FALSE]    # Subset TCGA counts to common genes
dsmz_counts <- dsmz_counts[common, , drop=FALSE]    # Subset DSMZ counts to common genes

# ---------------------- Build merged matrices & define batches ----------------
Xc_raw <- cbind(tcga_counts, dsmz_counts)           # Combine TCGA and DSMZ raw counts
batch <- factor(c(rep("TCGA", ncol(tcga_counts)), rep("DSMZ", ncol(dsmz_counts))))  # Define

# ---------------------- PCA BEFORE correction (baseline) ----------------------
M0_log <- logCPM(Xc_raw)                            # Compute log2-CPM for uncorrected data

# ---------------------- Batch correction (ONE place) --------------------------
cat("[INFO] Batch adjustment (global)...\n")        # Print message indicating batch correcti
method <- tolower(config$batch_adjust_method)       # Get batch correction method

if (method == "none") {                             # Check if no batch correction
  cat("[INFO] Skipping batch correction.\n")        # Print message
  M1_log <- M0_log                                  # Use uncorrected data
} else if (method == "combat_seq") {                # Check if ComBat-seq is selected
  if (min(table(batch)) < 2) {                      # Check for singleton batches
    cat("[WARN] Singleton batch detected; falling back to ComBat on logCPM.\n")  # Warn abou
    M1_log <- sva::ComBat(dat = as.matrix(M0_log), batch = batch,  # Apply standard ComBat
                          mod = NULL, par.prior = TRUE, prior.plots = FALSE, mean.only = FALS
  } else {
    Xc <- Xc_raw                                    # Use raw counts
    Xc[is.na(Xc)] <- 0                             # Replace NA with 0
    Xc <- round(Xc)                                 # Round to integers
    set.seed(42)                                    # Set seed for reproducibility
    Xc_adj <- sva::ComBat_seq(as.matrix(Xc), batch = batch)  # Apply ComBat-seq
    M1_log <- logCPM(Xc_adj)                        # Log-transform corrected counts
  }
```

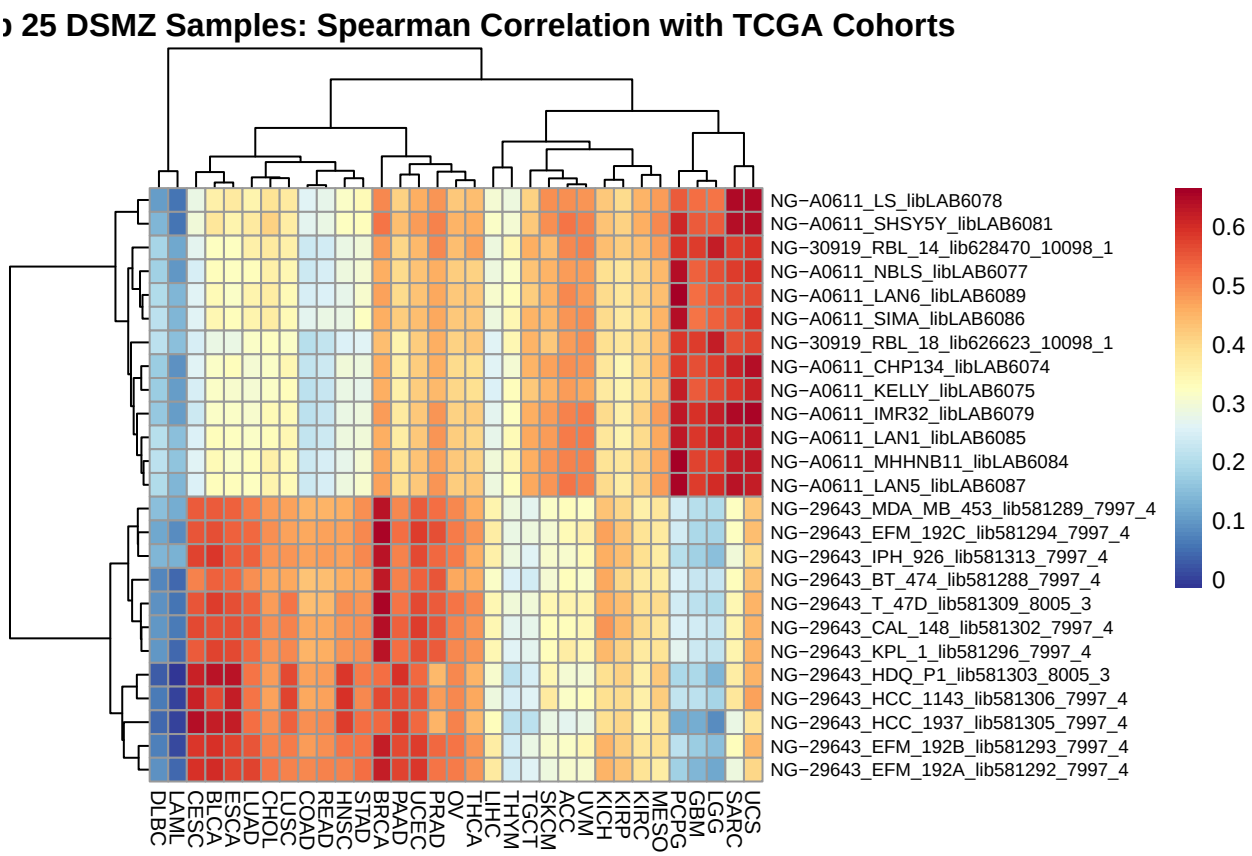
```
} else if (method == "combat") {                    # Check if standard ComBat is selected
  M1_log <- sva::ComBat(dat = as.matrix(M0_log), batch = batch,  # Apply standard ComBat
                        mod = NULL, par.prior = TRUE, prior.plots = FALSE, mean.only = FALSE)
} else {
  stop("[ERROR] Unknown batch_adjust_method. Use 'none', 'combat_seq', or 'combat'.")  # Stop
}

# ---------------------- Optional purity adjustment on TCGA (log scale) -------
purity <- load_purity_data(tcga$se, config$purity_tsv)  # Load purity data
if (!is.null(purity)) {                             # Check if purity data exists
  tcga_cols <- colnames(tcga_counts)                # Get TCGA sample columns
  tcga_log_adj <- purity_adjust_log(M1_log[, tcga_cols, drop=FALSE], purity)  # Adjust TCGA
  M1_log[, tcga_cols] <- tcga_log_adj              # Update adjusted TCGA data
}

# ---------------------- Correlation analysis (on adjusted log2-CPM) ----------
tcga_cols <- colnames(tcga_counts)                  # Get TCGA sample columns
dsmz_cols <- colnames(dsmz_counts)                  # Get DSMZ sample columns
tcga_log_mat <- M1_log[, tcga_cols, drop=FALSE]     # Subset TCGA log matrix
dsmz_log_mat <- M1_log[, dsmz_cols, drop=FALSE]     # Subset DSMZ log matrix
tcga_res <- compute_tcga_means(tcga$se, tcga_log_mat)  # Compute TCGA cohort means
tcga_means <- tcga_res$means                        # Extract TCGA means
stopifnot(identical(rownames(tcga_means), rownames(dsmz_log_mat)))  # Verify gene alignment
sel_genes_corr <- select_variable_genes(tcga_means, config$var_gene_set)  # Select variable
spearman <- compute_correlations(dsmz_log_mat, tcga_means, dsmz_meta, sel_genes_corr)  # Com

# Build effective mapping using external CSV if available
mapping <- build_effective_mapping(tcga_means, config$dsmz_meta_csv)  # Build organ and samp
dsmz_map <- perform_organ_mapping(dsmz_meta, spearman, tcga_means, mapping$organ2tcga, mappi
best_idx <- max.col(spearman, ties.method="first")  # Find index of maximum correlation
best_overall <- tibble(                             # Create summary of best correlations
  sample_id = rownames(spearman),                   # Sample IDs
  best_tcga_overall = colnames(spearman)[best_idx],  # Best TCGA cohort
  best_rho_overall = spearman[cbind(seq_len(nrow(spearman)), best_idx)]  # Best correlation
)
rho_allowed <- rep(NA_real_, nrow(dsmz_map))        # Initialize vector for organ-constrained
ok <- !is.na(dsmz_map$tcga_code)                    # Identify valid TCGA assignments
if (any(ok)) {                                      # Check if valid assignments exist
  for (i in which(ok)) {                            # Loop through valid assignments
    sample_id <- dsmz_map$sample_id[i]; tcga_code <- dsmz_map$tcga_code[i]  # Get sample and
    rho_allowed[i] <- spearman[sample_id, tcga_code]  # Extract correlation
  }
}
summary_tbl <- dsmz_map %>% select(sample_id, organ, tcga_code) %>%  # Create summary table
  left_join(best_overall, by="sample_id") %>% mutate(rho_allowed = rho_allowed)  # Join and

create_plots(summary_tbl, spearman, dsmz_map, config$outdir)  # Generate plots
```

```r
  create_discussion_plots(summary_tbl, spearman, dsmz_map, dsmz$meta, config$outdir)  # Genera
  save_results(spearman, summary_tbl, dsmz_map, config$outdir)  # Save results
  save_debug_info(tcga_counts, dsmz$raw, common, config$outdir)  # Save debug info

  # ---------------------- PCA before/after (log scale consistently) ------------
  tcga_lab <- sub("^TCGA-","", as.character(tcga_res$labels[colnames(tcga_log_mat)]))  # Clean
  tcga_df <- tibble(sample = colnames(tcga_log_mat), dataset = "TCGA", tcga_project = tcga_lab)
  proj_map <- tcga_project_to_organ_map(mapping$organ2tcga)            # Get TCGA-to-organ mapp
  tcga_df <- tcga_df %>% left_join(proj_map, by = "tcga_project") %>% mutate(organ = coalesce(o
  dsmz_df <- tibble(sample = colnames(dsmz_log_mat), dataset = "DSMZ", tcga_project = NA_charac
              left_join(dsmz_meta %>% select(sample_id, organ), by = c("sample"="sample_id")) %
              mutate(organ = coalesce(organ, "Other"))  # Replace NA organs
  sample_df <- bind_rows(tcga_df, dsmz_df)           # Combine metadata
  sel_genes_pca <- select_variable_genes(M0_log, config$var_gene_set)  # Select genes for PCA
  M0_log_pca <- M0_log[sel_genes_pca, , drop=FALSE]  # Subset uncorrected data
  M1_log_pca <- M1_log[sel_genes_pca, , drop=FALSE]  # Subset corrected data

  cat("[INFO] PCA BEFORE batch adjustment...\n")     # Print message for pre-correction PCA
  r2_before <- pc_dataset_s_(M0_log_pca, factor(sample_df$dataset), top_n = 10, outdir = config
  scree_before <- make_scree_plot_(r2_before$pc, config$outdir, "before")  # Create scree plot
  r2_cov_before <- pc_covariate_r2_matrix_(pc = r2_before$pc, sample_df = sample_df,  # Compute
                                           covariates = c("dataset", "organ", "tcga_project"),
                                           n_pcs = 10, outdir = config$outdir, tag = "before",

  cat("[INFO] PCA AFTER batch adjustment...\n")      # Print message for post-correction PCA
  r2_after <- pc_dataset_s_(M1_log_pca, factor(sample_df$dataset), top_n = 10, outdir = config$
  scree_after <- make_scree_plot_(r2_after$pc, config$outdir, "after")  # Create scree plot
  r2_cov_after <- pc_covariate_r2_matrix_(pc = r2_after$pc, sample_df = sample_df,  # Compute
                                          covariates = c("dataset", "organ", "tcga_project"),
                                          n_pcs = 10, outdir = config$outdir, tag = "after", sh

  cat("\n**PC variance explained by dataset (R^2):**\n\n")  # Print header for R² comparison
  r2_comparison <- tibble(                           # Create R² comparison table
    Stage = c("Before Batch Correction", "After Batch Correction"),  # Stages
    PC1_R2 = c(r2_before$r2_table$R2[r2_before$r2_table$PC == "PC1"],  # PC1 R²
               r2_after$r2_table$R2[r2_after$r2_table$PC == "PC1"]),
    PC2_R2 = c(r2_before$r2_table$R2[r2_before$r2_table$PC == "PC2"],  # PC2 R²
               r2_after$r2_table$R2[r2_after$r2_table$PC == "PC2"])
  )
  print(r2_comparison)                               # Print R² comparison

  cat(sprintf("[INFO] Pipeline completed successfully. Results saved to: %s\n", config$outdir))
  list(spearman_scores = spearman,                   # Return key results
       summary_table = summary_tbl,
       dsmz_mapping = dsmz_map,
       r2_comparison = r2_comparison)
}
```

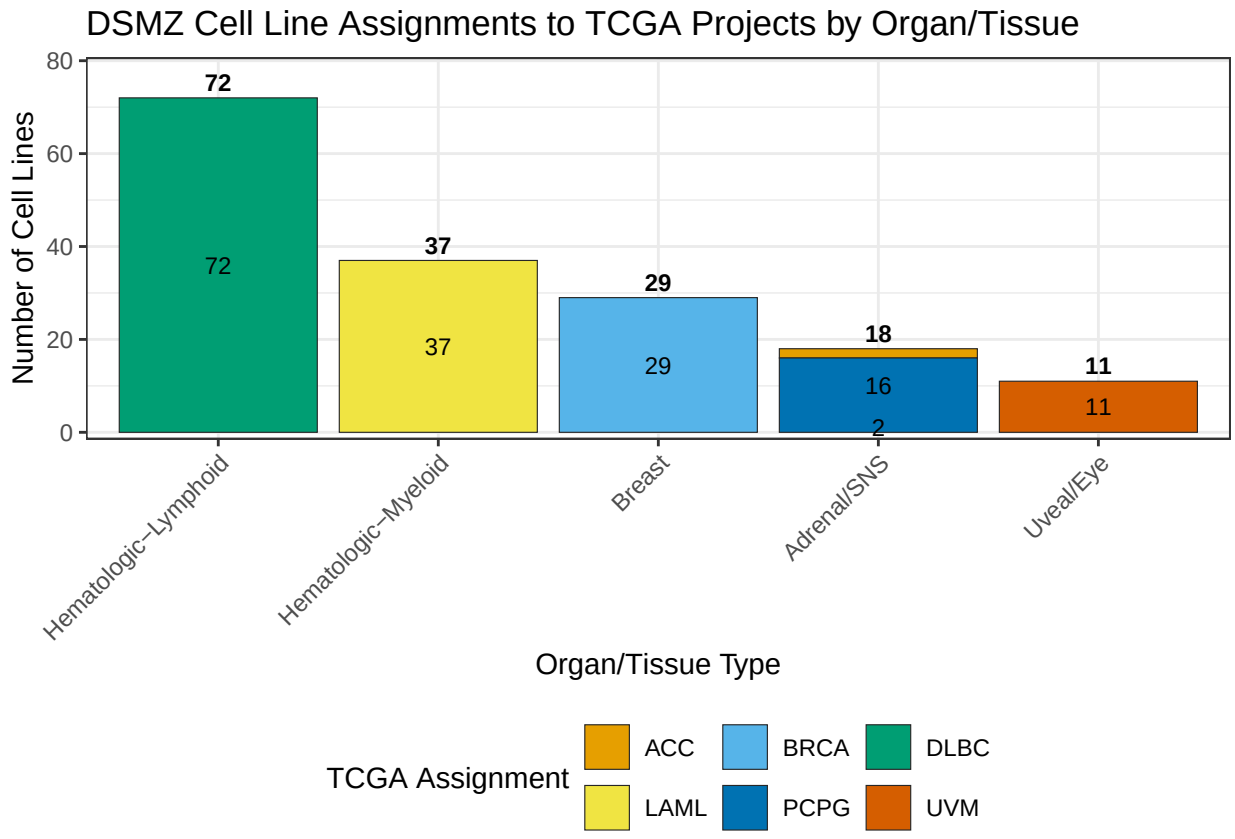
```r
# ============================================================================
# EXECUTION
# ============================================================================
if (!interactive()) {                                    # Check if running in batch mode
  results <- main()                                      # Run pipeline in batch mode
} else {
  cat("[INFO] Running in interactive mode. Call main() to execute pipeline.\n")  # Print messa
}
```

```
## [INFO] Starting pipeline...
## [INFO] Loading TCGA data...
## [INFO] TCGA dims: 60660 genes x 10272 samples
## [INFO] Loading DSMZ data...
## [INFO] DSMZ table: 60660 rows x 170 cols
## [INFO] Harmonizing TCGA gene IDs...
## [INFO] Building DSMZ count matrix and aligning metadata...
## [INFO] Building DSMZ count matrix...
## [INFO] Aligning metadata to counts...
## [INFO] Matched: 167 | Unmatched(meta): 0 | Unmatched(counts): 0
## [INFO] Saving aligned DSMZ data to cache: /home/chu25/data/dsmz/DSMZ_aligned_cache.rds
## [INFO] Shared genes: 60616
## [INFO] Batch adjustment (global)...
## Found 2 batches
## Using null model in ComBat-seq.
## Adjusting for 0 covariate(s) or covariate level(s)
## Estimating dispersions
## Fitting the GLM model
## Shrinkage off - using GLM estimates for parameters
## Adjusting the data
## [INFO] Using purity from SummarizedExperiment
## [INFO] Applying purity adjustment on log2-CPM...
## [INFO] Removing 878 purity-associated genes
## [INFO] Variable-gene setting: tcga_5k (5000 genes used)
## [INFO] Computing Spearman correlations...

## [INFO] Performing organ-constrained mapping (external-aware)…
## [INFO] Creating correlation plots...
```

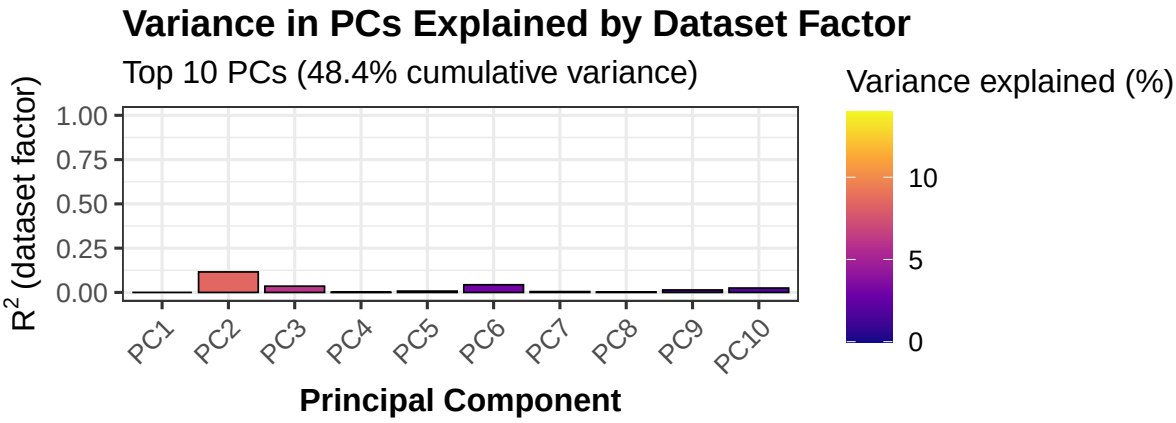

Distribution of Best DSMZ...TCGA Correlations by Organ/Tissue
Kruskal...Wallis p = 1.18e−21
Best Spearman ..
0.7
0.6
0.5
0.4
0.3
****
****
****
****
***
Adrenal/SNS
Breast
Hematologic−Lymphoid
Hematologic−Myeloid
Uveal/Eye
Organ/Tissue Type

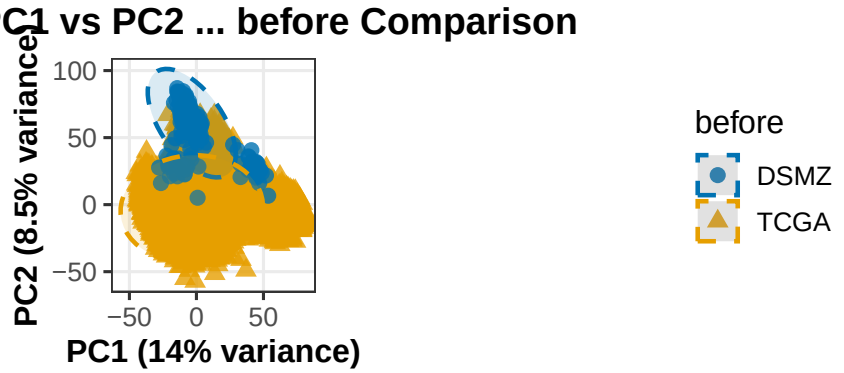


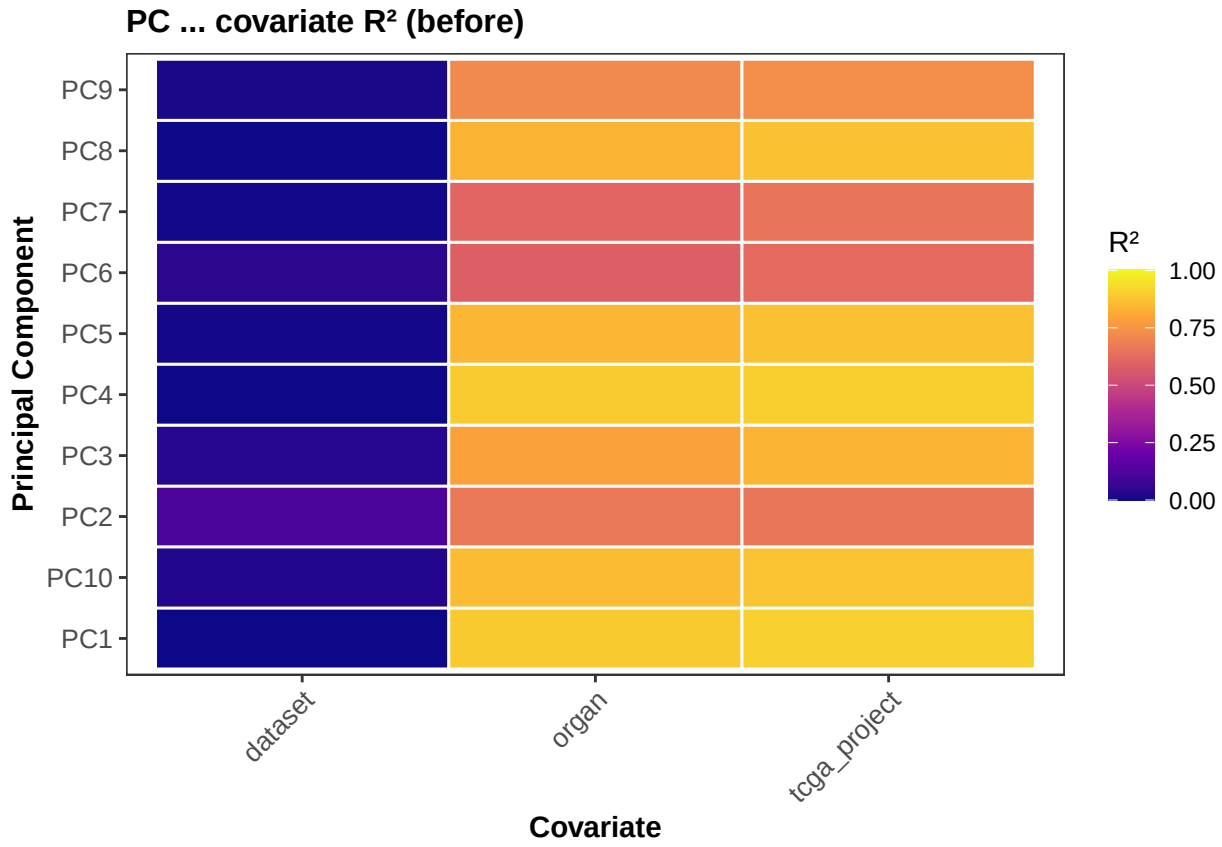

```
## [INFO] Plots saved to:  /home/chu25/dsmz/results/correlation5/figs
## [INFO] Creating 3-view discussion plots...
```



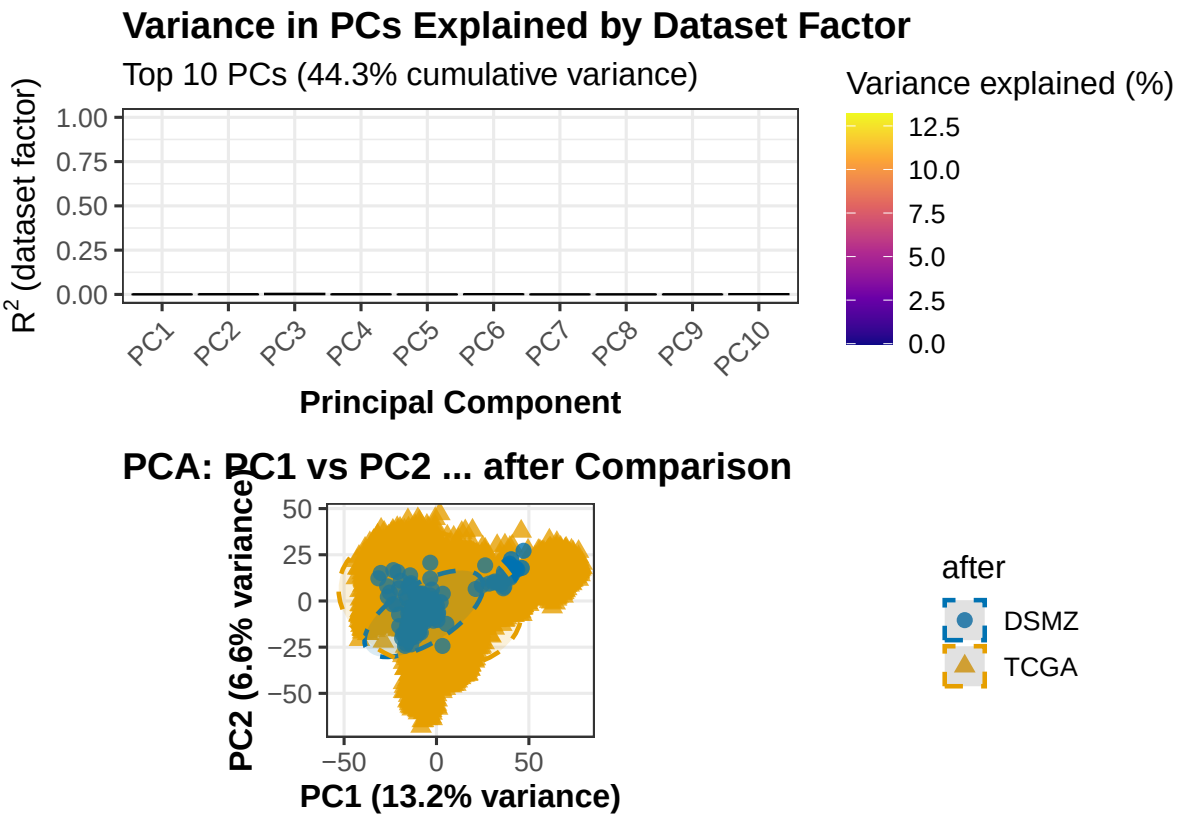

```
## [INFO] Discussion plots saved to:  /home/chu25/dsmz/results/correlation5/figs_discussion
## [INFO] Saving correlation tables...
## [INFO] Variable-gene setting: tcga_5k (5000 genes used)
## [INFO] PCA BEFORE batch adjustment...
```


Variance in PCs Explained by Dataset Factor
Top 10 PCs (48.4% cumulative variance)
R2 (dataset factor)
1.00
0.75
0.50
0.25
0.00
PC1
PC2
PC3
PC4
PC5
PC6
PC7
PC8
PC9
PC10
Principal Component
Variance explained (%)
10
5
0

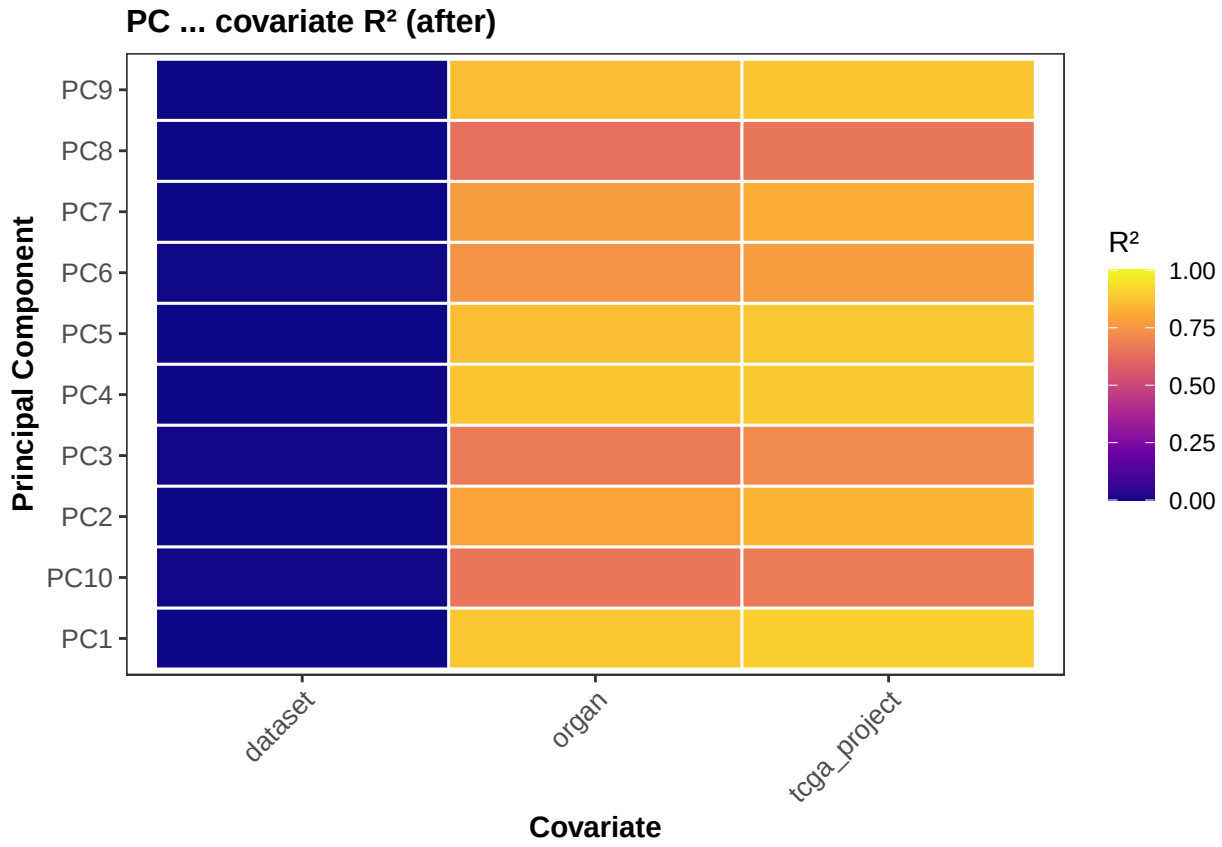

PCA: PC1 vs PC2 ... before Comparison
PC2 (8.5% variance)
100
50
0
−50
−50
0
50
PC1 (14% variance)
before
DSMZ
TCGA




```
## [INFO] PCA AFTER batch adjustment...
```


Variance in PCs Explained by Dataset Factor
Top 10 PCs (44.3% cumulative variance)
R2 (dataset factor)
1.00
0.75
0.50
0.25
0.00
PC1
PC2
PC3
PC4
PC5
PC6
PC7
PC8
PC9
PC10
Principal Component
Variance explained (%)
12.5
10.0
7.5
5.0
2.5
0.0

PCA: PC1 vs PC2 ... after Comparison
PC2 (6.6% variance)
50
25
0
−25
−50
−50
0
50
PC1 (13.2% variance)
after
DSMZ
TCGA




```
##
## **PC variance explained by dataset (R^2):**
##
## # A tibble: 2 x 3
##   Stage                       PC1_R2   PC2_R2
##   <chr>                        <dbl>    <dbl>
## 1 Before Batch Correction 0.00000411 0.116
## 2 After Batch Correction  0.000252   0.000730
## [INFO] Pipeline completed successfully. Results saved to: /home/chu25/dsmz/results/correlati
```